

3-Dimensional Matching

Karp #17 — Trabajo Grupal Optimización

Javier Farías Uriel Navarrete Vicente Abarca

Universidad Andrés Bello

Abril 2026

Problema: 3-Dimensional Matching (Karp #17)

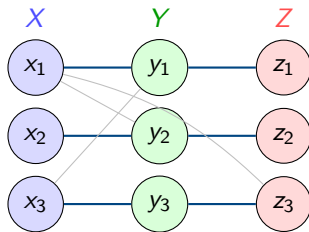
Datos:

- Tres conjuntos X, Y, Z con $|X|=|Y|=|Z|=n$
- Conjunto de tripletas $T \subseteq X \times Y \times Z$,
 $|T|=m$

Objetivo (max-3DM): Encontrar el mayor subconjunto $M \subseteq T$ tal que cada elemento aparece a lo sumo una vez en M .

Ejemplo ($n=3, m=5$):

- $M^* = \{(x_1, y_1, z_1), (x_2, y_2, z_2), (x_3, y_3, z_3)\}$
- $|M^*| = 3 = n$ (matching perfecto)



Modelo matemático (ILP)

Variable: $s_t \in \{0, 1\}$ para cada tripleta $t \in T$

Formulacion ILP

$$\text{máx} \quad \sum_{t \in T} s_t$$

$$\text{s.a.} \quad \sum_{t \ni i} s_t \leq 1 \quad \forall i \in X$$

$$\sum_{t \ni j} s_t \leq 1 \quad \forall j \in Y$$

$$\sum_{t \ni k} s_t \leq 1 \quad \forall k \in Z$$

$$s_t \in \{0, 1\}$$

Características:

- m variables binarias
- $3n$ restricciones lineales
- Relajacion LP **no** integra en general
- Decisión: $\text{OPT}(I) = n?$

Consecuencia:

La relajacion LP admite gap $\Rightarrow$ no basta redondear; se necesita **backtracking exacto**.

Complejidad: por que 3DM es NP-completo

Pertenencia a NP:

- Certificado: k índices de tripletas
- Verificador: 3 bitmasks en $O(n)$

NP-dificultad (Karp 1972):

Reduccion $3\text{-SAT} \leq_p 3\text{DM}$ via gadgets:

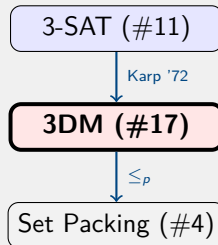
- **G-V**: codifica variables
- **G-C**: codifica clausulas
- **G-G**: recolecta basura

Instancia: $n=6q$ elems., $m=O(q^3)$ trips.

max-3DM: APX-difícil

- Sin PTAS salvo $P=NP$
- Mejor aprox.: $\approx 0,66$

Cadena de reducciones



La 3ª dimension rompe la unimodularidad que hace P al matching bipartito.

	BRUTE	SMART
Estrategia	Back. 2^m	Con heurísticas
Complejidad	$O(2^m)$	Exp.; poda
Bitmask	Si $O(1)$	Si $O(1)$
Cota sup.	No	$\min(fX, fY, fZ)$
MRV	No	Si
Fwd. check	No	Si

Lenguajes (misma logica):

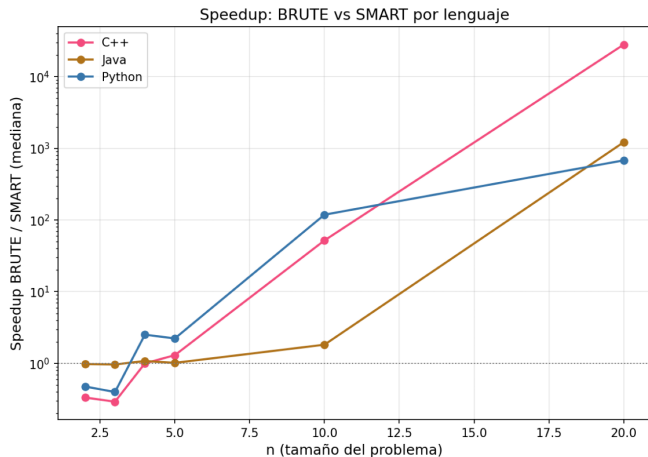
- **Python 3.11**: claridad
- **C++17**: rendimiento (-O2)
- **Java 17**: JIT HotSpot

Decision tecnica clave:

uint64/long bitmasks $\Rightarrow$ conflicto y popcount en $O(1)$.

Determinismo: misma semilla $\Rightarrow$ mismos nodos en los 3 lenguajes.

Resultados experimentales



Cifras destacadas:

SMART vs BRUTE

Speedup > **60 000**×
para $n=20$, $m=80$

Entre lenguajes (SMART, $n=30$)

C++ 18 ms

Java 52 ms

Python 1 150 ms

C++ vs Java: $\approx 3\times$

C++ vs Python: $\approx 63\times$

*Nodos explorados identicos en los 3 lenguajes
(37 854 para $n=30$, $m=120$).*

Lo aprendido:

- El modelo ILP guía directamente las podas de SMART.
- MRV + forward checking reduce el árbol de búsqueda de forma exponencial.
- Lógica idéntica en 3 lenguajes; solo cambia la velocidad.

Limitaciones:

- $n \leq 50$; instancias *large* ($n=100$) intratables.
- Sin paralelismo (un solo hilo).

Trabajo futuro

- **Aproximaciones:** local-search de Cygan ($\approx 0,66$).
- **FPT:** parametrización por treewidth.
- **Paralelismo:** work-stealing en backtracking.

**3DM es NP-completo,
pero las podas importan.**

Backup: Tabla de tiempos completa (small suite)

Lenguaje	Algoritmo	n	Mediana (ms)	Instancias
C++	brute	5	0.006	6
C++	brute	10	0.467	25
C++	brute	20	timeout	25
C++	smart	5	0.005	6
C++	smart	10	0.011	25
C++	smart	20	0.060	25
Java	brute	10	5.932	25
Java	brute	20	timeout	25
Java	smart	10	3.361	25
Java	smart	20	4.454	25
Python	brute	10	24.544	25
Python	brute	20	timeout	25
Python	smart	10	0.269	25
Python	smart	20	2.725	25

Timeout = 30 s (small suite). Valores 5000 ms en CSV corresponden al timeout del script.

Backup: Correctitud de las 6 implementaciones

Instancia	n	m	OPT	Acuerdo	Todos
mini1	2	3	2	SI	2
mini2	3	5	3	SI	3
noperf	3	3	1	SI	1
dense	4	16	4	SI	4
empty	5	0	0	SI	0

0 discrepancias de bug en todas las instancias completadas.

Los 6 solvers producen identico $\text{OPT}(I)$ y mismos nodos explorados.

Backup: Gadgets de la reduccion $3\text{-SAT} \leq_p 3\text{DM}$

Gadget de variable u_i con k_i ocurrencias:

- Elementos: $\{t_p^i\}_{p=1}^{k_i} \subset X$ (verdadero), $\{f_p^i\} \subset X$ (falso), $\{a_p^i\} \subset Y$, $\{b_p^i\} \subset Z$
- Tripletas: $T_p^i = (t_p^i, a_p^i, b_p^i)$ y $F_p^i = (f_p^i, a_{p \bmod k_i+1}^i, b_p^i)$
- Lema de consistencia: el matching fuerza O todos T_*^i O todos F_*^i

Gadget de clausula $C_j = (\ell_1 \vee \ell_2 \vee \ell_3)$:

- Dos nuevos elementos $c_j \in Y$, $c'_j \in Z$
- Tres tripletas-clausula (una por literal); exactamente una puede ser seleccionada

Tamano de la reduccion: $n = 6q$ elementos por dimension, $m = O(q^3)$ tripletas, construccion en tiempo polinomial.

Para $n=50$, $m=200$ (instancia *random_n50_m200_s0*):

- **C++ smart**: 39.0 s (38 944 ms)
- **Java smart**: 29.8 s (29 843 ms)
- **Python**: timeout en 60 s

Explicacion:

- Mismos nodos explorados (57 998 790): logica identica
- JIT HotSpot (Temurin-21) con warm-up completo supera GCC-O2
- `Long.bitCount()` llega al hardware POPCNT igual que C++
- JIT hace mejor especulacion de branches en el loop interno

Este fenomeno esta bien documentado: JIT puede superar AOT en workloads con patrones predecibles y hot loops.